paper_note.md

Exact and Lower-Bound Values for Right-Isosceles-Free Subsets of the Square Grid

Abstract. For a positive integer n, let a(n) be the largest size of a subset of the n×n integer grid that contains no three points forming a right isosceles triangle. We compute a(n) exactly for $n \le 7$ by constraint programming, confirm the known values for $8 \le n \le 11$, and report lower bounds for $12 \le n \le 16$. Erratum: OEIS A271906 already contains lower bounds for n = 12–15 (29, 32, 36, 38; R. Israel and G. Resta, 2016). Our bounds for 12–15 therefore add nothing new, and our values for 13–15 are weaker. Only n = 16 (value 40) is not listed there. We do not prove optimality for $n \ge 8$, and our upper bounds are loose.

1. Problem

A *right isosceles triangle* in $\mathbb{Z}^2$ is a triple {P, Q, R} with a right angle at P and |PQ| = |PR|. Equivalently, there exist a vector u = (a, b) ≠ (0, 0) and a point P such that the three points P, P+u, P+R(u) lie in the set, where R(a, b) = (−b, a) is rotation by 90°.

Let $S \subseteq \{0, \ldots, n-1\}^2$ and call S *admissible* if it contains no right isosceles triangle. Define

$a(n) = \max\{\, |S| : S \text{ admissible} \,\}$.

This is OEIS A271906, which lists a(1), …, a(11) = 1, 2, 4, 6, 9, 11, 14, 17, 20, 23, 26.

2. Method

Exact model. We introduce a Boolean variable for each grid point and, for each right isosceles triangle T in the grid, the clause “not all three points of T are chosen.” We maximize the number of chosen points with Google OR-Tools CP-SAT. The number of triangles grows from 4 (n = 2) to 6020 (n = 11).

Heuristic model. We use simulated annealing over subsets. The objective is the set size minus a penalty of λ times the number of triangles fully contained in the current set, with λ = 1.5. The move flips one grid point, and the temperature decreases geometrically from 1.0 to 0.02 over a fixed time budget. We record the largest admissible set found.

Independent verification. Every reported set is checked by a separate routine (`verify.py`). It enumerates all triples of points and tests the right-isosceles condition directly by dot products and squared lengths, without using the triangle list from the solver. The checker rejects a planted violation and agrees with the solver on all $n \le 9$.

3. Results

3.1 Exact values (proved by CP-SAT, optimality status reported by the solver)

n	a(n)	Status
1	1	optimal
2	2	optimal
3	4	optimal
4	6	optimal
5	9	optimal
6	11	optimal
7	14	optimal

These agree with OEIS A271906.

3.2 Lower bounds

n	best found	Source	Upper bound (CP-SAT)	Matches OEIS?
8	17	CP-SAT, annealing	19	yes (A271906)
9	20	CP-SAT, annealing	28	yes
10	23	CP-SAT, annealing	40	yes
11	26	CP-SAT, annealing	121	yes
12	29	CP-SAT (1500 s), annealing (150 s)	144	yes: a(12) >= 29 already in OEIS (R. Israel, 2016)
13	31	annealing (150 s)	—	weaker than OEIS bound a(13) >= 32 (G. Resta, 2016)
14	33	annealing (150 s)	—	weaker than OEIS bound a(14) >= 36 (G. Resta, 2016)
15	37	annealing (150 s)	—	weaker than OEIS bound a(15) >= 38 (G. Resta, 2016)
16	40	annealing (150 s)	—	not listed in the OEIS entry we checked

All sets in the table were verified by the independent checker. For n = 12, two different methods reached 29, but neither proved that 29 is optimal.

3.3 Example construction (n = 12, |S| = 29)

Rows are shown from top (y = 11) to bottom (y = 0); # marks a chosen point.

```
. # # # # # . . # . . .
. . . . . . # . . # . .
. . . . . . . # . . # .
. . . . . . . . . . . #
# . . . . . . . . . . #
. # # . . . . . . . . #
. . . . . . . . . . . #
```

```
. . # # . . . . . . . #
. . . . . . . . . . . #
# . . # . . . . . . . #
. . # . . . . . . . . #
# # . . . . . . . . . #
```

3.4 Status of exact values and density

- **a(8) = 17 is now proved** (CP-SAT with 2×2 block cuts, 93 s, status OPTIMAL). It matches OEIS, so it is not new.
- **a(12) is not proved.** After 3000 s the solver held a set of size 29 with upper bound 72, which is the trivial 2×2 bound. The solver gave no improvement on the trivial bound.
- **Attempt to prove a(12) ≤ 29 with window cuts.** Adding the constraint that every k×k window (k ≤ 8) holds at most a(k) points, with the proved values a(k), did not resolve the question. For target 30, CP-SAT returned UNKNOWN after 2400 s on 2 cores. The n = 9 target 21 was also UNKNOWN after 200 s.
- **Density.** Covering the n×n grid by k×k blocks gives limsup $a(n)/n^2 \le a(k)/k^2$. The best k we can compute (k = 7) gives 2/7. This bound is weaker than the literature: Károlyi and Solymosi (arXiv:2607.22828, July 2026) cite an upper bound $F(n) \ll n^2/(\log n)^{(1+c)}$ from Pilatte's extension of the Bloom–Sisask method, which implies the density tends to 0. They also give a lower bound $F(n) = \Omega(n^{1.3})$. Our block argument therefore adds nothing asymptotic.

4. Limitations

1. **Optimality is unknown for n ≥ 8.** The CP-SAT upper bounds (19, 28, 40, 121, 144) are far above the best sets found. The gap for n = 11 and 12 is large, so the lower bounds could be improved.
2. **Lower bounds for n ≥ 12 come from a single heuristic run per n (plus one CP-SAT run for n = 12).** Other seeds or longer runs may give larger values.
3. **Novelty check is limited.** We checked the OEIS entry for A271906 (which in fact contains lower bounds for n = 12–15) and one preprint (Károlyi–Solymosi, arXiv:2607.22828). The preprint's results concern asymptotics, not the small-n values in this paper. A published table for n ≥ 12 may exist, and we did not check further.
4. **No theoretical content.** We prove no general bound on a(n). Trivially, each 2×2 block of the grid holds at most two admissible points, since any three corners of an axis-aligned unit square form a right isosceles triangle. This gives $a(n) \le 2\lceil n/2 \rceil^2$, which is far from the computed values.

5. Reproducibility

Files are in `./`:

- `rit.py`: triangle enumeration and CP-SAT model.
- `local.py`: simulated annealing.
- `verify.py`: independent checker.
- `local_n{12..16}.json`, `big_n{11,12}.json`: reported sets.
- `summary.json`: verification results.

Run `python3 local.py N SECONDS SEED`, then `python3 verify.py`. The CP-SAT runs used 2 workers and time limits of 900 s (n = 11) and 1500 s (n = 12). The machine had 2 cores.